

Aufgabenstellung: Abstrakte Klassen in Java

(Klassen „Kreatur“, „Ritter“, „Drache“ sowie Arbeiten mit abstrakten Oberklassen und konkreten Unterklassen)

Ziel der Aufgabe

In dieser Aufgabe analysieren und entwickeln Sie ein Java-Programm, das eine einfache Klassenstruktur mit einer **abstrakten Oberklasse** und zwei **konkreten Unterklassen** modelliert.

Sie lernen dabei,

- wie abstrakte Klassen in Java verwendet werden,
- wie gemeinsame Eigenschaften in einer Oberklasse zusammengefasst werden,
- wie eine abstrakte Methode in Unterklassen konkret umgesetzt wird,
- wie sich gemeinsame und spezialisierte Merkmale unterscheiden,
- wie ein Testprogramm die Beziehungen zwischen Oberklasse und Unterklassen sichtbar macht.

Der Fokus liegt auf dem Verständnis, dass eine abstrakte Klasse als allgemeines Modell dient, von dem keine direkten Objekte erzeugt werden, während Unterklassen dieses Modell durch eigenes

Verhalten konkret ausfüllen.

Fachlicher Hintergrund

In der objektorientierten Programmierung werden Klassen häufig in Beziehung zueinander gesetzt.

Eine wichtige Möglichkeit ist die Verwendung von **abstrakten Klassen**.

Eine abstrakte Klasse beschreibt ein allgemeines Grundmodell. Sie kann gemeinsame Attribute enthalten und Methoden vorgeben, die in Unterklassen konkret umgesetzt werden müssen.

In dieser Aufgabe wird die Klasse `Kreatur` als allgemeine abstrakte Oberklasse verwendet.

Davon leiten sich die Klassen `Ritter` und `Drache` ab.

Typische Beziehungen in dieser Aufgabe sind:

- ein `Ritter` ist eine `Kreatur`,
- ein `Drache` ist eine `Kreatur`,
- beide besitzen gemeinsame Merkmale,
- beide unterscheiden sich aber in der konkreten Art des Angriffs.

Damit eignet sich dieses Beispiel gut, um den Unterschied zwischen einer **abstrakten Oberklasse** und **konkreten Unterklassen** zu verdeutlichen.

Vorgegebene Klassenstruktur

Im Package `abstrakt.template` sollen folgende Klassen entwickelt und verwendet werden:

- `Kreatur`
- `Ritter`
- `Drache`
- eine Klasse mit `main`-Methode zum Testen

Die Klassen haben dabei folgende Rollen:

- `Kreatur` ist die **abstrakte Oberklasse**,
 - `Ritter` und `Drache` sind **konkrete Unterklassen** von `Kreatur`,
 - die `main`-Methode dient zum Testen des Programms.
-

Aufgabe 1: Entwicklung der abstrakten Klasse „Kreatur“

Ziel

Sie entwickeln die Klasse `Kreatur` als abstrakte Oberklasse der Klassenstruktur.

Zu entwickelnde Klassenstruktur

Die Klasse `Kreatur` soll gemeinsame Merkmale enthalten, die für alle Kreaturen gelten.

Beispielhaft soll die Klasse mindestens enthalten:

```
private String name;
```

sowie passende Methoden, zum Beispiel:

```
public Kreatur(String name)
public abstract void angreifen()
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Kreatur` so, dass:

- jede Kreatur einen Namen besitzt,

- der Name im Konstruktor gesetzt wird,
- die Klasse als abstrakte Oberklasse dient,
- die Methode `angreifen()` als abstrakte Methode vorgegeben wird.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `abstract`,
- ein Attribut `name` vom Typ `String`,
- einen Konstruktor zur Initialisierung,
- eine abstrakte Methode `angreifen()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- welche Aufgabe eine abstrakte Oberklasse hat,
- warum eine abstrakte Klasse nicht direkt zur Objekterzeugung gedacht ist,
- wie gemeinsame Eigenschaften zentral zusammengefasst werden.

Aufgabe 2: Entwicklung der Klasse „Ritter“

Ziel

Sie entwickeln die Klasse `Ritter` als konkrete Unterklasse von `Kreatur`.

Zu entwickelnde Klassenstruktur

```
public class Ritter extends Kreatur
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Ritter` so, dass:

- sie die allgemeinen Merkmale der Klasse `Kreatur` übernimmt,
- sie die Methode `angreifen()` konkret umsetzt,
- bei einem Aufruf der Methode deutlich wird, dass der Ritter sein Schwert nutzt.

Technische Anforderungen

Verwenden Sie:

- `extends Kreatur`,

- einen Konstruktor,
 - `super( . . . )` zur Initialisierung geerbter Attribute,
 - eine Überschreibung der Methode `angreifen( )`.
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie eine konkrete Unterklasse aus einer abstrakten Oberklasse entsteht,
 - wie geerbte Merkmale übernommen werden,
 - wie eine abstrakte Methode in einer Unterklasse umgesetzt wird.
-

Aufgabe 3: Entwicklung der Klasse „Drache“

Ziel

Sie entwickeln die Klasse `Drache` als weitere konkrete Unterklasse von `Kreatur`.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Drache` so, dass:

- sie die allgemeinen Vorgaben der Klasse `Kreatur` übernimmt,
- sie die Methode `angreifen()` konkret umsetzt,
- bei einem Aufruf der Methode deutlich wird, dass der Drache Feuer spuckt.

Technische Anforderungen

Verwenden Sie:

- `extends Kreatur`,
- einen Konstruktor mit `super(...)`,
- eine eigene Implementierung der Methode `angreifen()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass mehrere konkrete Unterklassen aus derselben abstrakten Oberklasse entstehen können,
- wie dieselbe Methode in verschiedenen Unterklassen unterschiedlich umgesetzt wird,

- wie Spezialisierung durch Vererbung und Methodenüberschreibung entsteht.
-

Aufgabe 4: Analyse der Methodenüberschreibung

Ziel

Sie analysieren, wie eine abstrakte Methode in verschiedenen Unterklassen konkret umgesetzt wird.

Anforderungen

Funktionale Anforderungen

Untersuchen und entwickeln Sie die Klassen so, dass:

- die Methode `angreifen()` in der Oberklasse nur vorgegeben wird,
- die Unterklassen `Ritter` und `Drache` diese Methode unterschiedlich umsetzen,
- bei der Ausgabe deutlich wird, welche konkrete Kreatur aktiv ist.

Technische Anforderungen

Beziehen Sie sich auf:

- abstrakte Methoden,

- Methodenüberschreibung in Unterklassen,
 - unterschiedliche Ausgaben bei gleichem Methodennamen.
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- was eine abstrakte Methode ist,
 - warum Unterklassen diese Methode umsetzen müssen,
 - wie dadurch gemeinsames Grundmodell und konkretes Verhalten zusammenwirken.
-

Aufgabe 5: Entwicklung der Testklasse mit main-Methode

Ziel

Sie entwickeln ein Testprogramm, das die Klassenstruktur praktisch demonstriert.

Zu entwickelnde Klasse

Die Testklasse soll eine `main`-Methode enthalten.

Anforderungen

Funktionale Anforderungen

Das Testprogramm soll:

1. ein Objekt der Klasse `Ritter` erzeugen,
2. ein Objekt der Klasse `Drache` erzeugen,
3. für beide Objekte die Methode `angreifen()` aufrufen,
4. die Unterschiede der Ausgaben auf der Konsole sichtbar machen.

Technische Anforderungen

Verwenden Sie:

- eine `main`-Methode,
 - mehrere Objekterzeugungen,
 - Methodenaufrufe auf Objekten verschiedener Klassen,
 - sinnvolle Konsolenausgaben.
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie konkrete Unterklassen im Programm verwendet werden,
 - wie eine abstrakte Klassenstruktur getestet wird,
 - wie sich unterschiedliches Verhalten in der Ausgabe zeigt.
-

Aufgabe 6: Analyse der Klassenhierarchie

Ziel

Sie analysieren die Beziehungen zwischen der abstrakten Oberklasse und den konkreten Unterklassen.

Anforderungen

Funktionale Anforderungen

Erklären Sie,

- warum `Kreatur` als abstrakte Oberklasse geeignet ist,
- warum `Ritter` und `Drache` von `Kreatur` erben,
- warum die Oberklasse nicht direkt instanziiert werden sollte,

- welche Vorteile diese Modellierung hat.

Technische Anforderungen

Beziehen Sie sich auf:

- abstrakte Oberklasse und konkrete Unterklasse,
- gemeinsame Merkmale,
- Wiederverwendung von Code,
- Spezialisierung von Verhalten.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass abstrakte Klassen ein allgemeines Modell beschreiben,
- warum gemeinsame Eigenschaften nicht mehrfach programmiert werden müssen,
- wie ein Modell durch abstrakte Klassen übersichtlich und erweiterbar bleibt.

Lernziele

Nach Abschluss dieser Aufgabe können Sie:

- erklären, was eine abstrakte Klasse in Java ist,
- den Unterschied zwischen abstrakter Oberklasse und konkreter Unterklasse beschreiben,
- Klassen mit `extends` modellieren,
- Konstruktoren mit `super( . . . )` verwenden,
- abstrakte Methoden in Unterklassen überschreiben,
- eine einfache Klassenhierarchie in einem Testprogramm anwenden.

Didaktische Einordnung

Diese Aufgabe erweitert grundlegende Java-Kenntnisse um das Konzept der **abstrakten Klassen** und der **Methodenüberschreibung**.

Sie knüpft an folgende Themen an:

- Klassen und Objekte
- Attribute
- Methoden
- Konstruktoren
- Vererbung

- `extends`
- `super(...)`
- abstrakte Klassen
- abstrakte Methoden
- Methodenüberschreibung
- Oberklassen und Unterklassen

Die Aufgabe zeigt, dass abstrakte Klassen eine gemeinsame Grundlage für verschiedene konkrete Klassen bilden können.

Sie bildet eine wichtige Grundlage für weiterführende Themen wie:

- Polymorphie
- dynamische Bindung
- Interfaces
- objektorientierter Softwareentwurf

Reflexions- und Kontrollfragen

Kontrollfragen zu abstrakten Klassen

1. Was ist eine abstrakte Klasse in Java?

2. Warum kann `Kreatur` nicht direkt instanziiert werden?
 3. Welche Aufgabe hat die Oberklasse `Kreatur`?
 4. Welche Aufgabe haben die Unterklassen `Ritter` und `Drache`?
-

Kontrollfragen zur Klassenstruktur

5. Warum erben `Ritter` und `Drache` von `Kreatur`?
 6. Welche Gemeinsamkeit besitzen beide Klassen?
 7. Worin unterscheiden sich `Ritter` und `Drache`?
 8. Warum ist hier eine abstrakte Oberklasse sinnvoll?
-

Kontrollfragen zur Programmlogik

9. Welche Aufgabe hat `super ( . . . )` im Konstruktor?
10. Was ist eine abstrakte Methode?
11. Warum muss `angreifen ( )` in den Unterklassen umgesetzt werden?
12. Welche Vorteile bietet diese Modellierung?
